

Глубокий анализ проекта repo_sum

repo_sum – это комплексный инструмент для анализа исходного кода и генерации документации с помощью больших языковых моделей (LLM). Он включает систему семантического поиска по коду (RAG – *Retrieval-Augmented Generation*) и интеграцию с OpenAI GPT для составления отчётов. Архитектура проекта модульная, с разделением на основные компоненты: **ядро анализа**, **RAG-система** (поиск по коду), **пользовательский интерфейс**, и **удалённый сервис** на VM для трудоёмких операций. Ниже представлено подробное исследование каждого модуля и файла, включая их логику, архитектуру, входные/выходные данные, а также выявленные несоответствия и рекомендации.

Обзор и назначение проекта

Цель проекта: Автоматизировать анализ репозитория и генерацию документации. Система сканирует исходный код, делит его на логические чанки, семантически индексирует эти фрагменты с помощью эмбедингов, а затем использует LLM (GPT) для генерации выводов или ответов на основе найденного кода. В версии **0.5** (готовится переход на 0.6) акцент сделан на интеграции RAG-сервиса на отдельной VM и оптимизации производительности. По состоянию на **октябрь 2025** система находится в стадии подготовки релиза 0.6: проводится рефакторинг, масштабирование на уровне VM-бэкенда и бенчмаркинг производительности.

Высокоуровневая архитектура: Проект следует модели **“RAG-as-a-Service”** – выделенный сервис на удалённой VM выполняет ресурсоёмкие задачи семантического поиска, а клиентская часть взаимодействует с ним через API. Векторное хранилище реализовано на базе **Qdrant** (векторная БД), настроенной на CPU и использующей квантование для оптимизации памяти. Предусмотрен гибридный поиск – комбинация dense-векторного и sparse-поиска (напр. метод **SPLADE** для извлечения ключевых слов). Это повышает полноту результатов: dense-эмбединги обеспечивают семантический поиск, а sparse-подход – точное совпадение по терминам.

Основные этапы работы системы:

- 1. Сканирование репозитория:** Локальная часть (клиент) рекурсивно обходит файлы с исходным кодом. Для этого служит модуль `file_scanner.py`, который фильтрует файлы по заданным расширениям/исключениям. На выходе получается список файлов для анализа.
- 2. Чанкирование кода:** Модуль `code_chunker.py` разбивает каждый файл на логические фрагменты (чанки) – например, по классам, функциям или блокам кода. Стратегия chunking настраивается (в `settings.json` установлен режим `"chunk_strategy": "logical"`, минимальный размер чанка и пр.). Итог – набор чанков с метаданными: содержимое, тип (функция, класс и т.д.), имя файла и т.д.
- 3. Индексация чанков (семантическое индексирование):** Клиентская часть отправляет чанки на удалённый сервис для индексации. Для этого используется класс `RemoteVectorStore` (модуль `rag/remote_vector_store.py`), предоставляющий метод `index_documents()`. Он разбивает коллекцию чанков на батчи и через HTTP API отправляет их на VM (эндпоинт `/index` у FastAPI сервиса на порту 8000). На стороне VM запрос обрабатывается `IndexerService` (модуль `rag/indexer_service.py`), который:
 4. вызывает эмбеддер для батча текстов (получение эмбедингов);

5. сохраняет полученные векторные представления в векторное хранилище (Qdrant). После успешной индексации каждого батча сервис возвращает количество добавленных документов.
6. **Семантический поиск по коду:** После индексации система может отвечать на вопросы или осуществлять генерацию документации. При запросе (например, вопрос пользователя либо внутренняя задача) клиент вызывает `RemoteVectorStore.search()` или высокоуровневый `QueryEngine`. Вначале запрос конвертируется в вектор (через `RemoteEmbedder` или непосредственный вызов `/embeddings` на VM) – это реализовано в `rag/remote_embedder.py`. Затем выполняется поиск ближайших векторов в хранилище через эндпоинт `/search` (обрабатывается `SearchService` на VM). `SearchService` поддерживает гибридный поиск: он может комбинировать результаты векторного поиска с результирующими по ключевым словам. Например, используется алгоритм **Reciprocal Rank Fusion (RRF)** для слияния двух списков результатов и **MMR (Maximal Marginal Relevance)** для устранения дублирующихся фрагментов. Настройки поиска (в `QueryEngineConfig`) позволяют включать/выключать эти методы: `rrf_enabled`, `mmr_enabled` и др., а также задать порог релевантности и количество результатов.
7. **Генерация ответа (документации):** Найденные релевантные кодовые фрагменты (чанки) формируются в контекст. Ранее для этого существовал модуль `context.py` (и экспериментальный `context_prototype.py`), предназначенный для компоновки найденных чанков в единый контекст для LLM. Однако на текущий момент формирование контекста может выполняться непосредственно в `QueryEngine` или на уровне UI. Далее модуль `openai_integration.py` использует OpenAI API для генерации текста ответа или документации, передавая в prompt соответствующий контекст. `OpenAIIntegration` (возможно, класс внутри этого модуля) настроен на работу с увеличенным токен-лимитом (до 2048) и включает механизм автоповторов при ошибках (retry при `RateLimitError` и пр.). Таким образом, финальный вывод – сгенерированный GPT текст, например описание архитектуры, список найденных багов в коде, рекомендация и т.д., в зависимости от задачи.

Ниже рассмотрены детали реализации по каждому ключевому файлу проекта.

Основные модули (ядро приложения)

main.py – запуск анализа репозитория

Файл `main.py` – это основной входной скрипт для анализа. В его начале происходит настройка среды (например, установка кодировки `utf-8` для вывода) и импорт всех необходимых компонентов. Он загружает конфигурацию (в т.ч. из `settings.json` и переменных окружения через `config.py`) и инициализирует основные объекты для анализа репозитория.

Основная логика `main.py` likely включает следующие шаги: - **Получение списка файлов:** с помощью `FileScanner` (`file_scanner.py`) собираются пути исходных файлов для анализа. Фильтрация файлов может основываться на конфигурации (например, игнорирование `node_modules`, бинарных файлов и пр.). - **Chunking файлов:** через `CodeChunker` (`code_chunker.py`) каждый исходный файл разбивается на чанки. В конфигурации задан минимальный размер чанка и флаг `enable_fallback`, который указывает, делать ли резервное дробление, если логический парсер не может разделить большой файл. - **Индексирование чанков:** создается экземпляр RAG-системы для индексации. В ранних версиях проект напрямую работал с Qdrant (возможно, создавая `VectorStore` локально), но теперь применяется **Factory Pattern** для выбора реализации (локальная или удалённая). Вероятно, `main.py` вызывает что-то вроде `rag.create_vector_store(config)` – фабричный метод, возвращающий либо `RemoteVectorStore` (если работаем с удалённой VM), либо локальный `VectorStore`. Согласно

спецификации, Factory Pattern был введён, чтобы **автоматически выбирать между локальной и удалённой реализацией** на основе контекста выполнения (VM сервер или клиент). Таким образом, на клиенте `rag.create_vector_store` вернёт RemoteVectorStore, а на VM – VectorStore (или InMemoryVectorStore для оффлайн режима). - Получив объект VectorStore, `main.py` вызывает метод индексации, передавая список чанков. Здесь реализована обработка возможных исключений: модуль `rag/exceptions.py` определяет классы ошибок, например VectorStoreException, VectorStoreConnectionError и пр., которые могут возникать при неудачном подключении к хранилищу или других сбоях. `main.py` обрабатывает эти исключения, возможно через try/except, и логирует их через модуль `logging`. - **Генерация отчёта:** После успешной индексации `main.py` может вызывать функции генерации документации. Для этого он использует OpenAIIntegration. Вероятно, `main.py` формирует запрос к GPT с описанием репозитория, найденными ключевыми фрагментами кода и т.д. **Выходные данные** `main.py` – сгенерированные MD-файлы отчётов (сохранённые на диск) либо вывод в консоль/интерфейс. Например, `main.py` может создавать README.md или другие документирующие файлы на основе анализа.

Стоит отметить, что `main.py` сейчас также включает интеграцию RAG, чего не было в изначальной реализации. Комментарий в начале файла отмечает: *"Теперь включает RAG систему для семантического поиска по коду."* Это означает, что ранее генерация отчётов происходила без этапа семантического поиска, а теперь добавлено индексирование и поиск по векторному хранилищу, чтобы GPT мог опираться на контекст кода.

Технические особенности: `main.py` использует `asyncio` для вызова асинхронных операций безопасно. Поскольку Streamlit/UI или просто Python main – синхронная среда, при вызове асинхронных функций (например, RemoteVectorStore.index_documents, возвращающего coroutine) используется утилита `run_async_safe` из `event_loop_manager.py`. Это предотвращает ошибки наподобие "RuntimeError: event loop is closed" или "already running". Также `main.py` учитывает переменные окружения (.env) и настраивает глобальные параметры (число потоков PyTorch/OMP, берётся из ParallelismConfig в config).

unified_launcher.py – унифицированный запуск (Setup + Web)

Этот модуль представляет собой вспомогательный лаунчер, позволяющий выполнить как настройку удалённой VM, так и запуск локального интерфейса. Он реализует 2-шаговый воркфлоу: 1. **Настройка VM (шаг "setup"):** выполнение скрипта `vm_start.py` – он настраивает окружение на удалённом сервере. Судя по контексту, `vm_start.py` подготавливает VM для запуска RAG-сервиса (например, проверяет/устанавливает зависимости, запускает контейнер с Qdrant, запускает FastAPI сервис и модель Jina). В репозитории присутствует `VM_STARTUP_CONFIGURATION.md` и `VM_STARTUP_COMPREHENSIVE_GUIDE.md`, описывающие необходимые переменные окружения и параметры VM. Также `vm_start.py` возможно открывает SSH-соединение или API вызовы к VM (IP 10.61.11.54) для развертывания сервисов. 2. **Запуск локального приложения (шаг "start"):** выполнение `run_web.py`, который поднимает локальный веб-интерфейс (Streamlit) для взаимодействия с системой.

Если `unified_launcher.py` запущен с параметром `all` (по умолчанию), он последовательно выполняет оба шага. Внутри он, скорее всего, использует модуль `subprocess` для вызова этих скриптов и `time.sleep` / проверки готовности между ними. Скрипт принимает аргументы командной строки (`argparse` используется для параметров "setup", "start", "all").

Выход: Удобство такого лаунчера в том, что пользователю достаточно одной точки входа. После `setup` VM будет запущена с сервисами, а `start` откроет браузер со Streamlit UI. *UnifiedLauncher* обрабатывает возможные ошибки запуска (например, если VM недоступна или `vm_start.py` вернул ошибку – тогда он выведет лог или остановит процесс).

run_web.py – запуск веб-интерфейса (Streamlit)

Скрипт `run_web.py` предназначен для запуска Streamlit-приложения, определённого в `web_ui.py`. Вероятно, он парсит аргументы (например, порт, режим) и затем вызывает команду `streamlit run web_ui.py` через `subprocess` или `os.system`. Данный файл упоминается как "Helper script for launching the Streamlit web UI.". Таким образом, он служит прослойкой, чтобы не заставлять пользователя вручную вводить команду streamlit – достаточно `python run_web.py`. В Windows, возможно, предусмотрен `.ps1` или другое, но универсально через Python.

Особенности: Может устанавливать флаг `--server.port` или `--browser.gatherUsageStats` и другие опции Streamlit перед запуском. Также, если UI запускается впервые, `run_web.py` может проверять, установлен ли Streamlit и, если нет, выдавать инструкцию. Однако в репозитории есть `requirements.txt`, где, скорее всего, Streamlit уже указан, так что установка выполнена заранее.

web_ui.py – пользовательский интерфейс (Streamlit)

Этот модуль определяет интерактивный веб-интерфейс для пользователя. Используется библиотека **Streamlit** для отображения элементов. `web_ui.py` импортирует ключевые классы из проекта: скорее всего `RepositoryAnalyzer` (если оформлен как класс) или напрямую функции из `main`, а также компоненты RAG (`VectorStore`, `QueryEngine`, `IndexerService`, `SearchService` и т.д.).

Основные функции `web_ui.py`: - **Отображение формы загрузки репозитория/выбора папки:** Вероятно, UI позволяет указать путь к анализируемому репозиторию или выбрать из списка (возможно, предыдущих). - **Кнопка "Индексировать":** запускает процесс индексации. При нажатии, UI вызывает функции из `main.py` или напрямую вызывает `FileScanner -> CodeChunker -> RemoteVectorStore.index_documents`. В Streamlit это может быть реализовано через `st.button` и соответствующую callback-функцию. Поскольку индексация может занять значительное время (сотни чанков, минуты), UI должно отображать прогресс. Вероятно, используются `st.progress` и сообщения о статусе (например, "Индексировано X/Y чанков"). - **Отображение результатов индексации:** После завершения индексации UI выводит сообщение об успехе (например, "Индексация завершена, добавлено N чанков за T секунд"). В коде действительно был пример подобного вывода: *"Indexed 0/260 chunks in 257.07s"* – этот лог указывал на баг, но формат такой строки присутствует. - **Поле ввода запроса (поисковый запрос или вопрос):** Пользователь может ввести вопрос о коде (например, "Какие основные классы в этом репозитории и их ответственность?"). При отправке, UI вызывает `SearchService/QueryEngine`: берёт текст вопроса, вызывает `remote embedder` для эмбединга запроса, потом `remote vector store` (поиск) или `QueryEngine`, получает список релевантных фрагментов. - **Отображение результатов поиска:** Найденные фрагменты кода отображаются, возможно, с контекстом – названием файла, частью кода, оценкой релевантности. Streamlit может использовать `st.code` для красивого вывода кода. Если применяется ранжирование, может быть выведено в порядке убывания релевантности. - **Генерация ответа LLM:** После поиска UI может автоматически или по нажатию

кнопки отправить запрос в OpenAI GPT, включив найденные фрагменты. Результат (сгенерированный текст) отображается как Markdown через `st.markdown`.

Технические моменты: UI должен учитывать состояние сессии Streamlit. Например, большие объёмы данных (списки чанков) можно хранить в `st.session_state` между нажатиями. Есть упоминание, что при баге был `NameError: chunk_type is not defined` в `web_ui.py` – вероятно, UI пытался отображать поле `chunk_type` чанка, но переменная не была доступна. Этот баг был исправлен (добавлена передача или определение `chunk_type`). Streamlit работает асинхронно, но не полностью поддерживает Python asyncio – возможно, RAG-вызовы выполняются синхронно с точки зрения UI (Streamlit запускает them in a separate thread or just awaits if `run_async_safe`). Чтобы UI не завис, время выполнения операций контролируется: например, in *HOTFIX_TIMEOUTS.md* отмечено, что при зависании поиска (отсутствие timeout) UI просто «висит». Поэтому важно, что все remote вызовы ставят таймаут.

file_scanner.py – сканирование файловой системы

Модуль отвечает за сбор всех файлов, подлежащих анализу. Он, вероятно, рекурсивно обходит директорию репозитория, применяя фильтры: исключение определённых папок (`.git`, `__pycache__`, например), ограничение по размерам или типам файлов. Возможно, `file_scanner.py` реализует функцию `scan_repository(path) -> List[Path]`.

Вход: корневая директория репозитория, возможно список включаемых/исключаемых шаблонов (например, через `settings.json` можно задать `"include_extensions": [".py", ".js", ...]` или `"exclude_dirs": ["node_modules", ...]`).

Выход: список файлов (путей), отсортированных или сгруппированных по языкам.

Он может также вычислять размер и предупреждать, если репозиторий слишком большой. Но основная логика – пройтись по дереву каталогов.

code_chunker.py – разбиение кода на чанки

Этот модуль реализует **chunking** – разделение текста исходного кода на логические фрагменты. Цель – выделить самодостаточные фрагменты (например, функции, классы, логические блоки) для дальнейшего индексирования, вместо сырого разбиения по строкам.

Как это может работать: - Открывает файл исходника, читает содержимое. - В зависимости от языка/синтаксиса применяет правила разбиения. Возможно, имеются парсеры (отдельные модули, которые были исключены из предоставленных файлов) для разных языков. Например, для Python – разделение по определениям функций и классов; для JavaScript – по `export` или `function`; для markdown/docs – по заголовкам. В упоминании пользователя было, что парсеры неинтересные – вероятно, они были вынесены и не включены. - Если *logical* стратегия (как указано в `settings.json`), то chunker старается разделить по синтаксическим границам. Если активирован `enable_fallback: true`, при сбое логического парсера (например, синтаксическая ошибка или неподдерживаемый формат) выполняется резервное разбиение – возможно, просто по длине или по пустым строкам через определённый интервал. - Каждый чанк формируется как структура (возможно, словарь или объект): `{'content': <текст>, 'file_path': ..., 'file_name': ..., 'chunk_name': ..., 'chunk_type': ...}`. В `memory_vector_store.py` видно пример подобной структуры с полями `content`, `chunk_type`, etc., где `content` мог быть `'placeholder'` для заглушек. В нормальном случае `content` содержит код, `chunk_type` – тип (например, `"function"`, `"class"`), `chunk_name` – имя функции/

класса если есть, `file_path` – полный путь файла, `file_name` – имя файла. - **Выход:** список таких чанков. Размер чанков контролируется: `min_chunk_size: 100` (символов) задан в `settings.json`, то есть слишком мелкие куски могут объединяться или отбрасываться.

Технически, `CodeChunker` может использовать регулярные выражения или встроенные парсеры AST. Отсутствие сложных парсеров в репозитории намекает, что используется комбинация простых эвристик. Возможно, `chunker` поддерживает несколько языков (в `settings.json` есть `languages_priority`: ["python", "javascript", "java"] и др.). Это может означать, что для указанных приоритетных языков есть специфические правила.

openai_integration.py – взаимодействие с OpenAI API

В этом модуле инкапсулируется логика вызова OpenAI GPT модели для генерации текста. Он может содержать класс (например, `OpenAIIntegration` или `OpenAIClient`) с методами для отправки промптов и получения ответов.

Особенности, выявленные в коде: - Увеличенный токенный лимит: в комментарии указано "поддержка полного текста отчёта, токенный лимит ↑ до 2048". Вероятно, параметры API (например, `max_tokens` или модель) настроены для генерации более длинных ответов. Возможна поддержка модели GPT-4 или аналогичной, чтобы уместить большой контекст (чанки кода). - **Petry-политика:** В коде видно, что класс инициализируется с параметрами `attempts`, `delay` и кортежем `retryable_exceptions` – включая `openai.RateLimitError`. Это означает, что реализован механизм повторных попыток: при `RateLimit` или временных ошибках API запрос повторится заданное число раз с задержкой. Это важно, так как интеграция с LLM должна быть устойчивой к транзитным ошибкам. Модуль, скорее всего, импортирует библиотеку `openai` (OpenAI Python SDK). - **Функциональность:** Основная функция – метод, принимающий промпт (или список сообщений для ChatGPT) и возвращающий завершённый ответ. Может быть реализована поддержка **статуса и прогресса**: например, хэширование запроса, чтобы избежать повторных API вызовов (кеширование) – в коде импортируется `hashlib`, возможно для этого. - **Вход/выход:** На вход принимает сгенерированный контекст (например, топ-5 релевантных чанков + вопрос). На выходе – текст ответа (который затем сохранится или отобразится). В проекте `OpenAIIntegration` может также записывать полученный ответ вместе с исходным контекстом для аудита (например, в лог или JSON), но явных указаний на это нет.

config.py – конфигурация и параметры

Модуль `config.py` задаёт все глобальные настройки через `dataclass`'ы. Он читает файл настроек (`settings.json`) и переменные окружения (`.env`) и формирует объекты конфигураций: - **VectorStoreConfig:** Параметры подключения к Qdrant: хост, порт, флаги gRPC vs REST, имя коллекции, размерность векторов и параметры индекса (HNSW M, ef, квантование). Интересно, что размерность вектора берётся не жестко, а вычисляется: сначала из ENV `EMBEDDING_DIMENSION`, иначе из `EMB_TRUNCATE_DIM` или `EMB_DIM`, иначе по умолчанию 1024. Это обеспечивает согласованность: если на этапе эмбединга модель обрезает векторы, то коллекция создаётся с такой же размерностью. - **EmbeddingConfig:** (В выводе выше не было, но судя по структуре `RagConfig` должно быть). Здесь задаются параметры эмбединга: вероятно, имя модели (например, "jina-embedding-v3"), тип модели (SentenceTransformer vs FastText), нужна ли нормализация векторов и т.д. Возможно, также максимальная длина текста для эмбединга и способ усечения (`truncate` vs `split`). - **QueryEngineConfig:** Настройки поискового движка – уже упомянутые флаги `max_results` (по умолчанию 10), `rrf_enabled` (Reciprocal Rank Fusion),

`use_hybrid` (включать ли гибридный поиск), `mmr_enabled` (Maximal Marginal Relevance пост-обработка) и параметр `mmr_lambda` для степени разнообразия. Также есть параметры кэша результатов (`cache_ttl_seconds`, `cache_max_entries`) – возможно, система кэширует ответы поискового движка для повторных запросов в течение 5 минут, чтобы не делать повторные вычисления. - **ParallelismConfig**: Количество потоков для PyTorch (torch), OpenMP, MKL. Установлено по умолчанию 4, чтобы при CPU-инференсе модель эмбединга не забирала все ядра (что важно для одновременного обслуживания запросов на VM). - **RemoteServiceConfig**: Настройки удалённого RAG-сервиса – URL хоста (10.61.11.54) и порта (8000), пути эндпоинтов (`/embeddings`, `/search`, `/index`, `/health`). Здесь же важные параметры: - `timeout_seconds: 3600` – таймаут ожидания ответа от удалённого сервиса. Это критический фикс: ранее стояло 600с (10 минут), чего оказалось недостаточно для больших батчей (512+ чанков). 10 минутный предел приводил к обрыву соединения при индексировании очень крупных проектов (такой случай описан в док-те **TIMEOUT_FIX_512_CHUNKS.md**). Теперь таймаут увеличен до 1 часа, что устраняет преждевременный разрыв соединения при долгой индексации. - `max_retries: 5` и `retry_delay: 10.0` – параметры повторных попыток. Их увеличили с исходных 3 попыток и 2 секунд задержки (в **HOTFIX_TIMEOUTS.md** это указано как хотфикс). Это сделано, чтобы более надёжно пробовать повторно запрос при временных проблемах сети или сервисе. - **SparseConfig**: Параметры sparse-поиска – указан метод "SPLADE" по умолчанию. Значит, система настроена использовать SPLADE для получения sparse-векторов документов (эта техника генерирует взвешенный набор токенов, увеличивая семантическую подсветку ключевых слов). Вероятно, в `sparse_encoder.py` реализовано получение SPLADE-вектора для текста (через заранее обученную модель). Пока что, возможно, используется ограниченно или находится в разработке (в коде не обнаружено явного вызова SPLADE, но закладка есть).

Все эти dataclass'ы собираются в **RagConfig**, и функция `RagConfig.from_dict` заполняет их из загруженного JSON (или .env). В итоге, при старте приложения, один объект `config: RagConfig` распространяется по системе, содержащий все настройки для RAG и связанных компонентов.

utils.py – вспомогательные утилиты

Модуль `utils.py` содержит общие вспомогательные функции, используемые по всему проекту. Нередко в таком модуле бывают: - Функции для форматирования вывода (например, преобразование секунд в человеко-читаемый формат времени). - Логические утилиты, например, сравнение версий, или объединение путей. - Возможно, функции для отладки, например, печать структуры данных (дерева каталогов) или вывод примеров чанков.

Конкретно в коде `utils.py` из результатов поиска видны вызовы `requests` и `session.post`, что указывает: **utils** может содержать обёртку над HTTP-запросами. Вероятно, там реализован метод вроде `post_with_timeout(url, data)` – отправка HTTP POST с установленным таймаутом. Если `utils.py` действительно отвечает за HTTP вызовы, он может быть задействован в `RemoteVectorStore` или `RemoteEmbedder`. Например, если не использовали aiohttp, то utils мог содержать код типа:

```
session = requests.Session()
resp = session.post(url, json=payload,
    timeout=config.remote_service.timeout_seconds)
```

Однако, судя по другим модулям, `RemoteEmbedder` и `TransportClient` используют `aiohttp` (асинхронно), а `RemoteVectorStore` возможно использует `requests` синхронно. Если так, `utils.py` может предоставлять `session` или функцию для этого.

Кроме HTTP, `utils` мог бы иметь: - Функции `safe_bool`, `safe_int` (но они прямо в `config.py` определены). - Логгер-настройки (но логгер настраивается, кажется, прямо в `config`).

В любом случае, `utils.py` не содержит ключевой логики RAG, а предоставляет общие части, что облегчает поддерживаемость. Например, `utils.py` наверняка используется в тестах или скриптах (в `scripts/` упоминания могут быть).

прочие скрипты и вспомогательные файлы

Проект содержит ряд скриптов в папке `scripts/` и документов в `docs/` и `rules/`, кратко упомянем их значение: - **`vm_start.py`** – скрипт настройки VM. Вероятно, подключается по SSH к удалённой машине, экспортирует необходимые переменные окружения (`.env`), запускает Docker-контейнер с Qdrant (если используется Docker), и затем запускает FastAPI приложение (например, через Uvicorn, модуль `vm_rag_service.py`). Также, возможно, он загружает модель эмбединга (Jina) – на что указывает папка `.github/workflows/ci.yml` и presence of huggingface caches. `vm_start.py.backup` – резервная копия прежней версии, оставлена для сравнения. - **`vm_rag_service.py`** – непосредственно FastAPI приложение, работающее на VM. В его начале перечислены эндпоинты: `/embeddings`, `/search`, `/index`, `/health`. Он запускает event loop (FastAPI/uvicorn), и импортирует Factory для получения компонентов. Например, внутри него: - при старте определяет единожды объекты: `embedder = Factory.create_embedder()` (получит CPUEmbedder с моделью Jina), `vector_store = Factory.create_vector_store()` (получит локальный Qdrant VectorStore). - Эндпоинт `/embeddings`: принимает список текстов, возвращает их эмбединги (векторные представления). Реализовано, скорее всего, через вызов `embedder.embed(texts)` и возврат массива. - Эндпоинт `/index`: принимает список документов (чанков) для индексации. В теле запроса – JSON с массивом объектов (`content`, `metadata`). Обработчик `/index` вызывает `IndexerService.index(documents)` и возвращает, например, количество успешно проиндексированных. - Эндпоинт `/search`: принимает запрос (текст или вектор) и опциональные фильтры, возвращает топ-N результатов (список чанков с метаданными и оценками). Здесь вызывается `SearchService.search(query)` – внутри него сначала эмбединг запроса (через `embedder`), потом поиск в `vector_store`, потом ранжирование. - Эндпоинт `/health`: простой проверочный, возвращает, например, строку "ok" или информацию о состоянии (может, свободная память). `vm_rag_service.py` также может содержать interceptors: например, обработку исключений (CircuitBreaker – если постоянно фейлится эмбединг, возвращать 503), логирование запросов, контроль памяти (в OOM-репортах упоминается, что при 99% RAM срабатывал Circuit Breaker, поэтому, возможно, `vm_rag_service` перед каждым запросом проверяет свободную память и при нехватке не выполняет тяжёлую операцию). - **`collect_vm_metrics.py`** – скрипт, скорее всего, для метрик VM: собирает и сохраняет показатели (CPU, RAM, время ответа) во время работы. Может использоваться для бенчмаркинга performance benchmarking (упоминается в плане релиза 0.6). - **`scripts/migrate_to_jina_v3.py`** и **`database_migration_jina_v3.py`** – скрипты миграции. Судя по названию, ранее использовалась другая модель или хранение, и нужно было перенести данные/конфигурацию на новую модель Jina v3. Скрипты могли конвертировать старые эмбединги или коллекции Qdrant под новое размерное пространство (например, если старая модель имела векторы 768d, а новая 1024d). - **`scripts/check_timeouts`** – набор скриптов (`py`, `ps1`, `sh`) для проверки установки таймаутов. Видимо, в ответ на проблему зависания, были написаны утилиты, которые проверяют, что во всех вызовах `requests/HTTP` выставлен `timeout`. `check_timeouts.py` может парсить репозиторий и

искать `requests.post` без `timeout` или `asyncio.wait_for` обёртки. Это указывает, как серьёзно отнеслись к проблеме зависаний. - `rules/` – эта директория содержит “память системы” – документы с планами, отчётами о багфиксах, техническими долгами и архитектурой. Например, `Project Overview.md`, `Technical Architecture.md`, `Development Roadmap.md`, `Technical Debt.md`. Эти файлы помогают соотнести код с замыслами: - `Project Overview` (Обзор проекта) – общие цели, прогресс, что планируется к релизу 0.6. - `Technical Architecture` – подробный техсправочник по состоянию системы (на 23.09.2025). Там подтверждены ключевые моменты: использование `Jina v3` (570M параметров модели) на VM, модульная архитектура (`Core/RAG/Parsers/UI/Testing`), гибридный поиск `dense+sparse`. - `Development Roadmap` – план развития, содержит пункты по интеграции VM, оптимизации, подготовке к релизу. Упомянуто слияние ветки `jina-embeddings-v3` в `master`, что говорит об активной разработке вокруг новой модели эмбедингов. - `Technical Debt` – список технического долга (на 03.10.2025). Там перечислены области рефакторинга: VM backend, тестирование, конфигурация, `chunking`. Ответственный указан Claude, что видимо внутреннее имя AI-помощника, помогавшего вести документацию. В `Technical Debt` документе, вероятно, описаны несоответствия между кодом и лучшими практиками (например, “остались неиспользуемые модули `context_prototype`, требуется удалить”, “дублирование кода в `memory_vector_store` и `vector_store`, надо унифицировать” и т.п.). - `BUGFIX_REPORT_2025_10_06.md` – отчёт о багах после индексации от 6 октября 2025. Очень важный документ: он фиксирует 3 критичных бага и их решение: 1. Баг 1: Чанки не сохранялись в Qdrant (0 из 260 проиндексировано) – тот самый случай, когда после индексации отобразилось “Indexed 0/260 chunks”. Симптомы: эмбединги вроде бы прошли успешно (показало завершение батча 256 чанков и ещё 4 чанков), но при сохранении в хранилище что-то пошло не так, и в итоге 0 записей. - Причина: Ошибка в `indexer_service.py`. Код определял функцию индексирования через `getattr(self.vector_store, 'index_documents')` и проверял `asyncio.iscoroutinefunction(index_fn)`. Хотя `VectorStore.index_documents` определена как `async` (в `vector_store.py:486`), проверка `coroutinefunction` вернула `False`, потому что, видимо, метод привязанный (`bound method`) не считается корутиной. Поэтому условие шло не по той ветке и вызов делался через `await asyncio.to_thread(index_fn, points)`. Это неверно – вызывать асинхронную функцию через `to_thread` нельзя, и в итоге индексация фактически не выполнялась. Исправление заключалось в корректной идентификации `coroutine` и вызове `await index_fn(points)` напрямую. После фикса (вероятно, использовали `iscoroutinefunction(index_fn)` or `iscoroutine(index_fn(points))`) или проще – сделали отдельные методы для `sync/async`). 2. Баг 2: `NameError: chunk_type` не определён в `web_ui.py`. Симптомы: при попытке отобразить `chunk_type` (видимо, в таблице результатов или логах) переменная отсутствовала в области видимости. Исправили, видимо, передав `chunk_type` в нужную функцию или удалив этот вызов. 3. Баг 3: Ошибка “object of type 'coroutine' has no len()” на VM-сервисе. Это обычно возникает, когда пытаются измерить длину корутины, не дождавшись результата. Возможно, где-то в `search_service` или `query_engine` собирали, например, `len(results)` до того, как `results` получены (если `results` был `coroutine`). Либо JSON-сериализация попыталась посчитать `len`. Исправление – явно дождаться выполнения корутины (`await`) прежде чем работать с объектом. Все три бага отмечены как исправленные и протестированные, что соответствует тому, что 7 октября был релиз `Factory Pattern` и, вероятно, эти баги уже не воспроизводятся. - `HOTFIX_TIMEOUTS.md` (02.10.2025) – описывает ситуацию, когда `CircuitBreaker` срабатывал из-за `timeout` при высокой загрузке RAM (99% и `swarp thrashing`). Решение: увеличить таймауты (как мы видим, сделали 3600с) и, возможно, подстроить параметры `CircuitBreaker`. - `RAG_TIMEOUT_FIX_COMPREHENSIVE_PLAN.md` – план комплексного исправления проблем с таймаутами в RAG. Вероятно, объединяет: увеличение таймаутов, оптимизация пакетной отправки, улучшение асинхронности. - `RECURSION_FIX_FACTORY_PATTERN_SPEC.md` (07.10.2025) – подробная спецификация внедрения `Factory Pattern` для устранения проблемы рекурсии. Здесь описано, как фабрика определяет контекст (VM или клиент) и возвращает соответствующие реализации компонентов. Проблема

рекурсии заключалась в том, что без фабрики VM-сервис при индексации мог рекурсивно вызвать сам себя: например, `IndexerService` на VM по незнанию мог использовать `RemoteVectorStore`, что снова шлёт запросы на тот же VM, вызывая замкнутый круг. Теперь фабрика на VM возвращает локальный `VectorStore`, и индексация выполняется локально, разрывая цикл. - `Development Roadmap.md`* – упоминает масштабирование и оптимизацию (например, поддержка двух моделей эмбедингов) – этого явно нет текстом, но раз пользователь указал, это может быть их идея сверх плана.

В совокупности, наличие этих документов свидетельствует, что кодовая реализация активно подтягивается к обозначенным требованиям, но есть места, где планы и код временно расходятся (см. раздел технического долга ниже).

Теперь, подробно рассмотрим критические модули RAG-системы, так как они являются сердцем семантического поиска и требуют особого внимания.

Модули RAG (Retrieval-Augmented Generation)

Модуль `rag/` содержит компоненты, реализующие хранение и поиск эмбедингов, а также фабрику для выбора между локальными и удалёнными реализациями. Ниже разбор каждого файла и их взаимодействия.

factory.py – фабрика RAG-компонентов (реализована)

Файл `rag/factory.py` вводит **Factory Pattern** для создания основных RAG-компонентов: `Embedder` и `VectorStore`. Цель – **автоматический выбор локальной или удалённой реализации** в зависимости от среды выполнения.

В реализации присутствуют: - Перечисление контекста выполнения, вероятно `ExecutionContext` (например, `VM` vs `CLIENT`). - Функция `detect_execution_context()`: возможно, проверяет наличие определённых признаков, например, окружная переменная `RUN_IN_VM` или IP хоста. В `vm_start.py` при запуске сервиса могли устанавливать что-то вроде `os.environ["EXECUTION_CONTEXT"] = "VM"`. Либо проверка: если хост (в `RemoteServiceConfig`) равен `"localhost"` или `"0.0.0.0"`, то мы на VM. В любом случае, эта функция задаёт глобальный контекст. - Класс `Factory` с статическими методами: `create_embedder(config)`, `create_vector_store(config)`. Каждый из них внутри делает:

```
if context == ExecutionContext.VM:
    # вернуть локальную реализацию
else:
    # вернуть удалённую реализацию
```

Например, `create_vector_store` на VM вернёт экземпляр класса `VectorStore` (определённого в `vector_store.py`), а на клиенте – `RemoteVectorStore` (`remote_vector_store.py`). То же с эмбеддером: на VM – `CPUEmbedder` (`embedder.py`), на клиенте – `RemoteEmbedder` (`remote_embedder.py`). - Также `Factory` может содержать логику синглтонов: т.е. он может кэшировать созданные объекты, чтобы повторно не создавать (особенно это актуально для `embedder`, потому что загрузка модели – операция тяжёлая, модель должна быть единственным экземпляром). - Устранение рекурсии: Как описано, если `IndexerService` на VM вызовет `Factory.create_vector_store`, он получит **локальный** `VectorStore`, а не `Remote`, и тем самым не будет

слать HTTP самому себе, а сразу запишет в Qdrant. Аналогично, SearchService на VM при Factory.create_embedder получит локальный эмбеддер (Jina), а не RemoteEmbedder (который бы делал HTTP, что тоже не нужно).

Вход: объект конфигурации (RagConfig) или необходимые части (например, Factory.create_vector_store может принимать VectorStoreConfig).

Выход: экземпляр нужного класса (уже инициализированный).

Благодаря фабрике, остальная часть кода может вызывать унифицированно `Factory.create_X` без заботы о том, где она исполняется. В документации указано, что Factory Pattern реализован и протестирован, и деплой на VM проведён, что подтверждает успешное внедрение.

factory_prototype.py – прототип фабрики (устаревший)

Этот файл представляет ранний набросок фабричного подхода. Судя по названию и тому, что есть тест `test_factory_prototype.py`, **factory_prototype** – экспериментальный или предыдущий вариант реализации. Он, вероятно, содержал схожие идеи (выбор лок/remote), но мог быть неполным или менее элегантным. Например, *FactoryPrototype* мог требовать явной передачи флага `is_remote`.

Похоже, после успешной реализации нового `factory.py`, прототип больше не используется в основном коде (поиск по проекту показал упоминание factory_prototype только в его тесте) – т.е. это “мёртвый код”, сохраняемый временно для справки. Его можно удалить в будущем, чтобы не путать разработчиков.

embedder.py – локальный эмбеддер (CPUEmbedder)

Модуль содержит класс эмбеддера, работающего на CPU на стороне VM. Он отвечает за превращение текста (чанка кода или запроса) в векторное представление фиксированной размерности (по умолч. 1024). Основные черты: - **Модель эмбеддинга:** Используется либо фреймворк **Jina** (v3) с моделью ~570M параметров, либо библиотека **Sentence Transformers**. В комментарии упоминается поддержка FastEmbed и Sentence Transformers. Возможно, **FastEmbed** – это internal название для ускоренного pipeline (возможно, Jina’s Optimized Embedding). - **Инициализация:** При создании `CPUEmbedder` он загружает модель (из интернета или локального кэша). Это может быть, например, модель типа `all-MiniLM-L6-v2` или специализированная под код (CodeBERT). В *Development Roadmap* есть намёк на переход на Jina v3 – видимо, модель Jina v3 (название условное, может быть “embedding model v3”) стала использоваться. - **Метод `embed(documents: List[str]) -> List[vector]`:** Основной метод, возвращающий эмбеддинги. Он, вероятно, обрабатывает входы батчами (например, чтобы не переполнить память, делает `batch_size=16/32`). В docstring к `vector_store.py` упомянута “*батчевая загрузка точек 512-1024*” – то есть оптимизировано под пакетную обработку, и embedder наверняка поддерживает передачу списка текстов. - **Выходные данные:** массив чисел (list of float) размерности `vector_size` (1024). Если используется SentenceTransformer, то вызывается `.encode(documents)`. Если Jina, то возможно DocumentArray embeddings. - **Truncation:** Для длинных кусков кода embedder должен отсекаать или усреднять, иначе модель может не принять огромный текст. Вероятно, предусмотрено усечение до максимальной длины (например, 256 токенов), что могло бы объяснить наличие `EMB_TRUNCATE_DIM` (вместо этого, может, `EMB_TRUNCATE_LEN` – но в config использовали TRUNCATE_DIM как размерность, видимо оговорка).

Применение: `Embedder` используется на VM: - `IndexerService` вызывает `embedder.embed(chunk_texts_batch)` перед сохранением векторов. - `SearchService` вызывает `embedder.embed([query_text])` для перевода текстового запроса в вектор. Обе сервисные функции работают асинхронно, поэтому `embedder.embed` может быть обычной синхронной (CPU-bound) функцией. Если она синхронная, то для ее вызова внутри `async def` используют `await asyncio.to_thread(self.embed, docs)`, чтобы не блокировать event loop. Если же `embedder` имеет асинхронный интерфейс (например, `aiohttp` или `Jina async`), то просто `await`.

embedder_protocol.py – протокол/интерфейс эмбеддера

Судя по названию, здесь определён некий интерфейс (например, с помощью `Protocol` из `typing`). Возможно, `TransportClientProtocol` – интерфейс описывающий транспорт для удалённого эмбеддера. Это используется, чтобы `RemoteEmbedder` могла работать как с реальным HTTP-клиентом, так и с моками (например, при тестировании).

`embedder_protocol.py` может содержать: - Класс `BaseEmbedder` (abstract), определяющий метод `embed(texts)`. - Класс `TransportClientProtocol`: определяет необходимые методы транспорта (например, `async def post_embeddings(self, texts) -> List[vectors]`). `RemoteVMEEmbedder` (возможно класс из `remote_embedder.py`) будет реализовывать этот протокол с помощью, скажем, `aiohttp`. - Описания структур данных, типа `NamedTuple` для результата (`embedding + metadata`) – но маловероятно, скорее просто интерфейсы.

remote_embedder.py – удалённый эмбеддер (клиент)

Этот модуль реализует класс, который на клиентской стороне обращается к удалённому сервису для получения эмбеддингов. Назначение – **заменить локальную загрузку моделей на HTTP-запрос**. Основные детали: - Использует `aiohttp` для асинхронного HTTP: в `remote_embedder.py` импортируется `TransportClientProtocol` и из `transport_client.py` – реализация на `aiohttp`. Вероятно, `RemoteEmbedder` при инициализации создаёт `TransportClient` (из `transport_client.py`), передавая ему URL хоста/порта из конфигурации. - Метод `get_embeddings(texts: List[str])`: формирует запрос POST на `{host}/embeddings` с JSON, содержащим список текстов. Используется `await transport_client.post_json(endpoint, payload)`. `TransportClient` внутри настроен с сессией `aiohttp` (создаётся в `get_shared_http_session()` в `event_loop_manager` возможно). - **Event loop management:** Класс `RemoteEmbedder` должен вызываться в синхронном коде (например, в `QueryEngine` на клиенте). Для этого, аналогично `RemoteVectorStore`, в нём методы сделаны синхронными вращивками: например, `def embed(texts): return run_async_safe(self._async_embed(texts))`. `run_async_safe` – из `event_loop_manager.py` – запускает асинхронную корутину в уже существующем или новом event loop и ждёт результат. Это предотвращает ошибки, когда `Streamlit` или `main` вызывают корутину вне `async`. - **Timeouts и ошибки:** Если VM недоступна или долго не отвечает, вызов завершается по таймауту (таймаут задаётся в `RemoteServiceConfig` – 3600с). Если случается `asyncio.TimeoutError` или `aiohttp.ClientError`, `RemoteEmbedder` должен обработать их: либо пробросить как собственное исключение (например, `RagException`), либо fallback. `CircuitBreaker` (см. ниже) может быть встроен: возможно, если несколько `embed`-запросов подряд таймаутятся, система временно отключает RAG, чтобы не тормозить приложение. - **Выходные данные:** `RemoteEmbedder` возвращает список векторов (таких же, как выдала VM). Он может также прикреплять placeholder при ошибке: но скорее, при неудаче он бросит исключение, и тогда UI может показать сообщение об ошибке (или в крайнем случае – вернёт пустой список).

transport_client.py – асинхронный HTTP-транспорт

В этом модуле реализован низкоуровневый HTTP клиент на базе `aiohttp`. Судя по докстрингу, его даже сгенерировал Codex AI. `TransportClient` вероятно: - Создаёт один глобальный `aiohttp.ClientSession` (через `get_shared_http_session()` в `event_loop_manager`, чтобы повторно использовать сессию и connection pool). - Предоставляет методы `async post(endpoint: str, json: dict)`, `async get(...)` и т.д. Возможно, у него указаны default headers, timeouts. - `TransportClientProtocol` (из `embedder_protocol.py`) определяет сигнатуры этих методов. `TransportClient` реализует их.

Наличие отдельного транспорта говорит о стремлении повторного использования: `RemoteEmbedder` и, возможно, `RemoteVectorStore` могли бы использовать общий `TransportClient`. Однако, `remote_vector_store.py` не импортирует `transport_client` (по нашим поискам), и похоже, написан отдельно (возможно, с `requests`). Это как раз одна из **несогласованностей**: два удалённых клиента реализованы разными способами. В идеале, `RemoteVectorStore` тоже следовало бы переделать на `aiohttpTransport`, или наоборот.

vector_store.py – локальное векторное хранилище (Qdrant)

Этот файл содержит класс `VectorStore`, который взаимодействует с Qdrant – высокопроизводительной векторной базой данных. Ключевые моменты: - **QdrantClient**: Импортируется официальный клиент `qdrant_client.QdrantClient` и модели конфигурации (`Distance`, `VectorParams` и т.п.). Скорее всего, при инициализации `VectorStore` создает `self.client = QdrantClient(host, port, prefer_grpc=...)` на основе конфигурации. - **Инициализация коллекции**: Если коллекция (например, "code_chunks") не существует, `VectorStore` может её создать. Параметры коллекции берутся из конфигов: размерность векторов, метрика расстояния (по умолч. косинусное расстояние), параметры HNSW (`M=24`, `ef_construct=128`) и квантование (тип "SQ" – Scalar Quantization, включено). Квантование позволяет существенно сократить объём памяти, пожертвовав небольшой точностью – актуально, т.к. модель 1024-мерная, а число чанков может быть большим. - **Методы CRUD**: - `index_documents(points: List[Dict]) -> int`: добавляет документы в коллекцию. Каждый элемент `points` – словарь с как минимум: `id` (идентификатор точки, возможно хэш от содержимого или индекс), `vector` (сам вектор длины 1024), и `payload` (дополнительные данные – например, текст чанка или метаданные). Qdrant позволяет сохранять `payload` – и здесь, вероятно, сохраняется `content` чанка, `file_path`, `chunk_type` и т.д. Если `payload` включает `content`, то поиск можно делать гибридный: Qdrant может фильтровать по ключам (например, искать только в конкретном файле) или использовать некоторый текстовый поиск (но у Qdrant прямого full-text нет, поэтому, возможно, `payload` для вручную комбинированного поиска). Реализация функции, вероятно, `async` (в bugfix говорилось `index_documents` в `vector_store.py` определён как `async def`). Он вызывает `self.client.upsert(collection, points=batch)` (или `.upload_collection`) и возвращает количество добавленных. В случае неудач – бросает `VectorStoreException`. Реализована и `retry`-логика: в docstring упоминалось "CRUD операции с `retry` логикой". Возможно, декоратор или цикл: до `max_retries` попыток, с `retry_delay` паузой (эти параметры заданы в конфиге `RemoteService`, но тут локально – могут быть свои или использовать те же). - `search(query_vector, top_k, filter) -> List[Dict]`: выполняет поиск похожих векторов. Вызывает `self.client.search(collection, query_vector, top=top_k, query_filter=filter, search_params={...})`. `filter` может использоваться для гибридного поиска: например, если задан `filter` по какому-то ключевому слову, Qdrant выполнит **наряду** с векторным поиском фильтрацию. Однако, более вероятно, что гибридность тут достигается на уровне `QueryEngine`: он может выполнить два поиска – чисто векторный и какой-

то другим способом (например, BM25 по content). Но, поскольку config `use_hybrid=true`, Qdrant 0.9+ имеет фичу "Hybrid search" – можно передать одновременно вектор и фильтр со скриптовым условием. Неясно, используется ли она, но наличие SparseConfig намекает, что да. Возможно, QueryEngine генерирует Sparse-вектор с весами через Splade и передаёт его как фильтр (в Qdrant есть тип payload "keywords" c scores). - Дополнительные: `delete_vector(id)` для удаления документа, `update_vector(id, new_payload)` для обновления и т.д., если нужно (в контексте, возможно, нечасто нужны – разве что пересканирование репо и нужно обновить). - `health_check()` – может посылать пустой запрос или ping к Qdrant, используется Endpoint `/health` на сервисе, возвращающий {"status": "ok"} если все хорошо.

VectorStore работает только на VM. Он не используется на клиенте прямо (кроме случаев оффлайн-режима). Все вызовы к нему обернуты фабрикой и сервисами.

memory_vector_store.py – in-memory хранилище (оффлайн режим)

Для случаев, когда Qdrant недоступен или когда не требуется полноценный поиск, реализован класс **InMemoryVectorStore**. Он хранит векторы просто в Python структурах (списках, словарях). Назначение: - **Оффлайн-режим**: Если пользователь хочет протестировать функционал без поднятия VM (например, маленький репозиторий), или в юнит-тестах (чтобы не зависеть от внешнего сервиса). - **Тестирование алгоритмов**: InMemoryVectorStore может служить заглушкой, возвращающей фиксированные результаты (например, для проверки ранжирования QueryEngine).

Реализация: - Имеет поля, например, `self.vectors = []`, `self.index = {}` (словарь id -> индекс в vectors, для быстрых удалений). - `index_documents(docs)` просто сохраняет vectors (если doc содержит уже вектор, а если нет – возможно, вызывает embedder локально; но вероятнее docs сюда приходят уже с векторами, т.к. IndexerService если работает offline, сначала сам сделает embed). - `search(query_vector, top_k)` – вычисляет расстояния до всех сохранённых vectors (например, косинусное или L2) вручную, сортирует и возвращает top_k результатов. Здесь *chunk контент*, если хранился, может быть использован для формирования placeholder. В коде `memory_vector_store.py` мы видели явно словарь с `'content': 'placeholder'` – возможно, InMemoryVectorStore при индексации, если контента нет, ставит `'placeholder'` вместо текста (например, чтобы не хранить длинные строки). Это может объяснить жалобу, что *поиск возвращает placeholder вместо реального текста* – если по какой-то причине использовалось InMemory вместо Qdrant, то контент там фиктивный. Однако, в нормальной работе placeholder не должен доходить до пользователя. - `delete(id)` – удаляет из списков. - Очевидно, InMemoryVectorStore не поддерживает persistent хранение – при перезапуске всё потеряется. Он предназначен только для временного использования.

remote_vector_store.py – удалённое векторное хранилище (клиент)

Этот класс – пара для VectorStore, работающая на клиентской стороне, отправляя запросы на VM. Он **заменяет прямое подключение к Qdrant** на HTTP-вызовы FastAPI сервиса. Основные черты: - **Методы**: Предположительно, `index_documents(documents: List[Dict]) -> int`, `search(query: str или vector) -> List[Dict]`, возможно `delete(id)`, и `health_check()`. - **Реализация индексации**: - Синхронный метод `index_documents` вызывает асинхронный `_async_index_documents` через `run_async_safe`. То есть внутри определено `async def _async_index_documents(points)`, а наружу – обычная функция. Это аналогично RemoteEmbedder. - `_async_index_documents` формирует HTTP-запрос. Возможно, здесь использован обычный `requests` через `asyncio.to_thread`, или aiohttp. Поскольку

TransportClient отмечен "для RemoteVMEEmbedder", возможно RemoteVectorStore ещё не переделан на aiohttp. Он может делать:

```
async def _async_index_documents(points):
    async with aiohttp.ClientSession() as session:
        resp = await session.post(f"{base_url}/index", json=points,
                                timeout=...)
```

либо использовать requests:

```
await asyncio.to_thread(requests.post, url, json=points, timeout=...)
```

Оба подхода имеют pros/cons. Если проект начал с requests, могли сначала сделать через to_thread. В BUGFIX 6 октября ошибка была с to_thread (index_documents был async, его в to_thread запускали неправильно). Вероятно, после фикса они упростили: сделали index_documents в RemoteVectorStore синхронным напрямую (с calls requests) или убедились что iscoroutinefunction работает. - **Отправка чанков батчами:** Если чанков много (> например 256), RemoteVectorStore может делить на части. В упоминании пользователя: *клиент создаёт 314 чанков, отправляет, VM получает 256/58 документов, но все тексты пустые* – это похоже, RemoteVectorStore разбил 314 на два батча: 256 и 58. VM получила два запроса. Почему тексты пустые? Возможно, при сериализации JSON возникла проблема с unicode или с тем, что content слишком большой и где-то очистился. Или, что вероятнее, IndexerService ожидал поле `content`, а JSON имел другое поле или пустую строку. Если RemoteVectorStore, например, передавал только векторы, а тексты опустил (placeholder), то VM не смогла сохранить payload. В коде memory_vector_store `placeholder` был, но откуда на VM пустые тексты – возможно баг при сериализации: 314 чанков – большой JSON (~мегабайты текста кода), мог вызвать обрыв или not properly read. Или concurrency: если RemoteVectorStore не ждал завершения первого запроса и начал второй – потенциально race condition или перепутались ответы. Согласно bugfix, пустые тексты не упоминаются, но 0/260 было из-за coroutine issue. А случай 314 может быть отдельный баг, возможно позже рассмотренный. Это указывает на **несогласованность протокола**: надо убедиться, что JSON правильно содержит контент. - **Метод поиска:** RemoteVectorStore вероятно имеет `search(query_vector)` или `search(query_text)`. Если запрос строковый, может внутри вызвать RemoteEmbedder или, более вероятно, запрос /search позволяет передать сырой текст и сервис сам эмбеддит. В `vm_rag_service.py` endpoints: `/search` – гибридный поиск по векторам, неясно принимает ли он уже готовый вектор или текст. Возможно, для простоты, `/search` принимает JSON `{"query": "text of query"}` и VM сама внутри эмбеддит (что проще, т.к. embedder уже на VM). Тогда RemoteVectorStore.search(text) просто делается POST /search с `{"query": text}`. - Было замечание: *"Поиск отправляет placeholder вместо реальных векторов/текста"*. Если `/search` ожидал получить vector, а RemoteVectorStore послал placeholder, то поиск не работает. Вполне вероятно, что раньше `/search` API проектировали для приёма векторов (вдруг хотели дать возможность фильтровать по кастомному вектору). Но реализовали отправку без них. Например, RemoteVectorStore мог отправлять `{"query": "<PLH>"}` или пустую строку. Это нужно проверить: возможно, багфикс или hotfix касался того, что в search-endpoint не передавался нужный параметр. - **Timeout:** На поиск тоже должен ставиться таймаут. В config `timeout_seconds = 3600` – применяется ко всем трём типам запросов. Однако, если RemoteVectorStore.search использует requests напрямую, а разработчики забыли указать `timeout=...`, то запрос может зависнуть вечно. Пользователь упоминал: *"Отсутствие timeout в session.post для /search – может зависнуть на sock_read"*. Это означает, что действительно вызов /search не имел таймаута. Исправить нужно, добавив

timeout=config.remote_service.timeout_seconds в этот запрос. - **Retry:** Подобно embedder, RemoteVectorStore должен иметь retries. Если при индексации получаем ошибку, можно повторить (до 5 раз). Это тоже, вероятно, реализовано (в config max_retries=5, используется либо внутри RemoteVectorStore, либо на более высоком уровне IndexerService, который, заметив исключение VectorStoreConnectionError, сам повторит пакет).

indexer_service.py – сервис индексации (на VM)

Этот модуль работает на стороне VM и осуществляет логику индексации пачки документов: - Определён класс `IndexerService` с методом, возможно, `async index(documents: List[Dict])`. Он вызывается обработчиком `/index` FastAPI. - **Алгоритм:** 1. Разбивка входных документов на подбатчи (например, по 64 или 128, в зависимости от оптимального batch для модели). В bugfix логе видно, что 260 чанков обрабатывались как 256 + 4, скорее всего `IndexerService` сам поделил: первый батч max 256, второй – остаток. 2. Для каждого подбатча: вызвать `embedder.embed(batch_texts)` – получить list of vectors. 3. Подготовить точки для VectorStore: добавить к каждому вектору его id и payload (payload – информация о чанке). Если content слишком большой, могут решить **не** сохранять полный текст в payload (чтобы экономить RAM). Вполне возможно, вместо исходного content сохраняется только мета: файл, имя функции, а content либо укорочен, либо заменён placeholder. Это объясняет, почему при поиске, даже если находит, контент мог оказаться пустым – он мог нарочно не храниться. Однако, вероятнее, content хотя бы частично хранится, иначе LLM не увидит код. Technical Debt документ мог упоминать "не хранить полный текст каждого чанка в памяти, а при выводе подтягивать с диска". Это хороший подход: хранить только ссылки, а не дублировать весь код в RAM/Qdrant. 4. Вызвать `await vector_store.index_documents(points_batch)`. Это либо прямой вызов Qdrant (локально), либо – если не учли фабрику – мог бы быть вызов RemoteVectorStore (что и вызвало рекурсию ранее). После фабрики, `self.vector_store` внутри `IndexerService` должен быть локальным VectorStore на VM. 5. Учитывать, что `index_documents` может быть sync или async. Bug 1 показал, что `vector_store.index_documents` было async, и вызов через `asyncio.to_thread` был ошибкой. Исправлено: скорее всего, `IndexerService` теперь *всегда* await-ит корутину, если она. 6. Подсчитать успешно добавленные (VectorStore обычно возвращает число). Суммировать по батчам. - **Выход:** Количество добавленных документов или подробный отчет. FastAPI endpoint, скорее всего, возвращает JSON {"indexed": N, "elapsed": T}. UI потом отображает "Индексировано N/total чанков".

Технические аспекты: `IndexerService` может использовать **EventLoopManager** или прямые `await`. Если `embedder.sync`, его вызывает с `to_thread` (в `CPUEmbedder embed`, если не `async`). `VectorStore` likely `async` (because uses network/grpc to Qdrant – `QdrantClient` might have `async` variants). Также, `IndexerService` мог иметь **CircuitBreaker** проверку: если mem usage слишком высока перед началом, или если время на batch > threshold – может прерывать процесс (открывать автоматический "переключатель"). Однако, `CircuitBreaker` скорее используется на более высоком уровне (в `embedder` или `search`).

search_service.py – сервис поиска (на VM)

Отвечает за обработку запроса `/search`. Функции: - `async search(query: str) -> List[Dict]`: принимает текстовый запрос. - Процесс: 1. **Embed запрос:** через `embedder`: `q_vec = await embedder.embed([query])` (list with one vector). 2. **Сбор кандидатов:** двояко: - Если `use_hybrid = true`, то, возможно, генерируется ещё sparse представление запроса: `SparseEncoder.encode(query)` возвращает например список ключевых слов или vector весов по термину. - Далее, parallel: один `await vector_store.search(q_vec)` получить dense результаты; второй – использовать другой

механизм (например, BM25 через Whoosh/Lucene) получить результаты по ключевым словам **или** если Qdrant поддерживает, использовать filter: возможно, QdrantClient имеет метод `search_batch` где можно указать both vector and some filter scoring. Но вероятнее отдельно. - Если `use_hybrid = false`, то просто dense. 3. **Слияние результатов:** Если два списка – применить RRF (Reciprocal Rank Fusion). Конфиг `rrf_enabled` определяет, активировать ли. Если true, SearchService делает ранжирование: каждому результату присваивается оценка на основе рангов в каждом списке и смешивается. Это даст итоговый список с комбинированным рейтингом. 4. **MMR rerank:** Если `mmr_enabled=true`, то перед выдачей выполняется пост-обработка Maximal Marginal Relevance – удаление близких дублирующих фрагментов, чтобы ответ покрыл больше разных частей кода. `mmr_lambda` определяет баланс между релевантностью и разнообразием. 5. **Форматирование результата:** SearchService может отформатировать каждый найденный документ: например, оставить только нужные поля. Скорее всего, он возвращает для каждого chunka: `file_path`, `chunk_type`, `content` (возможно усечённый), `score` (или ранг). Также может сразу объединить смежные чанки (например, если два соседних фрагмента из одного файла попали – по желанию, но, скорее всего, это делает ContextBuilder). - **Выход:** JSON с массивом результатов. FastAPI отдаёт его клиенту.

В `search_service.py` комментарий говорит о “форматировании результатов для удобного отображения” – возможно, включает объединение полей `file_path` + `line numbers` в одну строку, ограничение длины `content` для UI, и т.п.

query_engine.py – поисковый движок (клиентский, CPUQueryEngine)

Этот модуль, вероятно, предназначен для клиентской стороны, чтобы абстрагировать работу поиска. Может существовать класс `CPUQueryEngine` (судя по импорту в `web_ui`), реализующий логику, похожую на SearchService, но уже на стороне клиента, если тот захочет искать без удалённого VM (например, если весь RAG локально, что возможно). Однако, учитывая RAG-as-Service, возможно QueryEngine в основном просто проксирует запрос на удалённый SearchService: - Если клиент работает с VM, `CPUQueryEngine.search(query)` может делать `return remote_vector_store.search(query)`. - Если бы работал без VM (в офлайн режиме), `CPUQueryEngine` сам содержит компоненты `embedder` и `vector_store` (`MemoryVectorStore`) и выполняет ту же логику (`embed` -> `search` -> `rank`).

В текущей конфигурации, скорее, `CPUQueryEngine` служит прослойкой, но особо не нужен – UI может напрямую вызывать remote search. Однако импорт `CPUQueryEngine` в веб говорит, что он используется. Возможно, QueryEngine также берет на себя: - Кэширование запросов (согласно `QueryEngineConfig`: `cache_ttl_seconds=300`, `cache_max_entries=1000`). То есть, QueryEngine может иметь словарь кэша: запрос->результаты, очищающийся через TTL. Это полезно, если пользователь повторяет один и тот же вопрос, чтобы не гонять по сети. - Упрощение интерфейса: например, QueryEngine может принимать запрос на естественном языке, а если RAG не доступен (например, выключен), то вернет graceful fallback (например: "Семантический поиск временно недоступен").

Также, `QueryEngine` может инкапсулировать вызов LLM: скажем, метод `answer_query(query)` который внутри сделает `search`, затем `openaiIntegration.generateAnswer(query, results)`. Но это уже выходит за рамки чисто "поискового движка" – скорее функция уровня Application.

circuit_breaker.py – защитный отключатель (Circuit Breaker)

В распределённых системах **Circuit Breaker** применяется для предотвращения каскадных сбоев: при множественных ошибках внешнего сервиса, временно прекращаются попытки и возвращается сразу ошибка. Здесь `circuit_breaker.py` вероятно реализует такой механизм для RAG-сервиса: - Если, например, 3 запросов к VM подряд завершились таймаутом или сбоем, CircuitBreaker "открывается" – переводит систему в состояние, когда дальнейшие запросы к RAG сразу отклоняются (например, UI мгновенно отвечает "Семантический поиск недоступен"). После некоторого времени (полу-открытое состояние) можно попробовать снова. - CircuitBreaker может быть встроен в RemoteVectorStore и RemoteEmbedder. Например, при вызове `index_documents`, перед тем как делать HTTP, можно проверить `if circuit_breaker.is_open: raise ServiceUnavailable`. Если closed – попытаться и на основании результата либо увеличить счётчик ошибок (если fail) либо сбросить (если success). - Также, CircuitBreaker мог учитываться в VM: *HOTFIX_TIMEOUTS.md* указал, что при thrashing (99% RAM) CircuitBreaker срабатывал. Возможно, на VM перед выполнением индексации проверяли свободную память – если очень мало, открывали breaker, чтобы не начинать новую тяжелую задачу и не упасть по OOM. - Нашедшееся в HOTFIX: *"Circuit Breaker открывается из-за timeout при 99% RAM"* – значит, было, что тяжелый запрос (512 чанков) вызвал свопинг, он шёл >10 мин, произошёл timeout, и CircuitBreaker решил, что сервис не отвечает, и открылся. Это, вероятно, было неправильным срабатыванием (false positive). Решение – увеличить таймаут, чтобы breaker не срабатывал зря. - **Реализация:** может быть простая: глобальные переменные или singleton класс с атрибутами: state (closed/open/half-open), fail_count, threshold, timeout. Если fail_count > threshold -> open, start timer. After timeout -> half-open (позволить один запрос тестовый). Примеры pattern есть в aiohttp CircuitBreaker. - Где иницируется: возможно, CircuitBreaker создаётся в `event_loop_manager` или `transport_client`, и перед каждым remote вызовом проверяется.

event_loop_manager.py – управление asyncio-циклами

Этот модуль решает проблемы смешивания синхронного и асинхронного кода. Он предоставляет: - Класс `EventLoopManager` (singleton), который может запускать и останавливать event loop. - Метод `run_async(coro, timeout=None)`: запускает coroutine и ожидает результат, даже если вызывается из основного потока. Он может создавать отдельный поток с event loop для выполнения, либо использовать существующий глобальный loop, если нет конфликтов. - Функция `run_async_safe(coro, timeout=None) -> result`: эта утилита, импортируемая повсеместно, используется чтобы вызвать async функцию из sync контекста. Она внутри делает `EventLoopManager.get_instance().run_async(coro, timeout)`. Если coroutine бросает исключение, его нужно пробрасывать. - `get_shared_http_session()`: возможно, функция, создающая/возвращающая глобальный aiohttp.ClientSession. Она может запускаться внутри async цикла. `transport_client.py` упоминает `get_shared_http_ses` (видимо усеченное название). Вероятно, реализовано как `async def get_shared_http_session(): ...`. - Также, Manager следит за тем, чтобы не было конфликтов: например, Streamlit сам использует asyncio, но обеспечивает `asyncio.new_event_loop` для user code. EventLoopManager может определять: если уже есть запущенный loop в текущем треде, то корутину отправляет на исполнение в ThreadPool (to_thread). Если же нет – просто запускает asyncio.run.

Необходимость: Без этого, попытка делать `asyncio.run` внутри Streamlit иногда приводила к ошибкам, также main->remote calls вызывали ошибки "event loop is running". Тест `test_async_sync_fix.py` вероятно проверяет, что вызовы RemoteVectorStore/RemoteEmbedder работают без таких ошибок. Благодаря EventLoopManager, проблема решена: все async HTTP вызовы идут через единый механизм.

retry_policy.py – политика повторных попыток

В этом файле реализованы механизмы повторения при ошибках. Хотя config задаёт retries, конкретная реализация может быть: - Декоратор `retry` для функций: например, `@retry(max_retries=5, delay=2, exceptions=(Exception1, Exception2))`. Тогда при вызове, если пойман указанный exception, будет повтор. - Специальные функции, например, `retry_async(coro_fn, *args)` для асинхронных функций (с использованием `asyncio.sleep` между попытками). - Политики: линейная задержка (как задано 10.0с), либо экспоненциальная. - Эту политику применяют к remote вызовам: `index_documents`, `search`, `embed`. В коде `bugfix/hotfix` прямо говорилось "HOTFIX: больше попыток (было 3 -> стало 5) и задержка 10с вместо 2с". Значит, механизм уже был – просто поменяли параметры. Вероятно, у них был декоратор на вызовы, или внутри `RemoteVectorStore` при `Exception` автоматически повторяет.

exceptions.py – классы исключений RAG

Содержит определения исключений для единообразия ошибок. Например: - `RagException` – базовый класс для всех ошибок RAG-системы. - `VectorStoreException`, `VectorStoreConnectionError` – для проблем с хранилищем (например, Qdrant недоступен). Импортируются и используются в `main` и `web_ui`, чтобы ловить специфичные проблемы. - `EmbedderException` – проблемы на этапе эмбединга (модель не смогла построить вектор, или timeout). - `CircuitBreakerOpen` – исключение, которое бросается, если `CircuitBreaker` не разрешает операцию (вместо выполнения). - И, возможно, `TimeoutException` если хотели явным отличать таймауты (но можно использовать `asyncio.TimeoutError` прямо). - В `bugfix`, Bag 1, наверняка был `VectorStoreException`, но не был пойман, из-за чего UI увидел 0/260. После исправления, видимо `main` или UI умеют отловить `VectorStoreException` и вывести более понятное сообщение.

vm_diagnostics.py – диагностика VM

Этот модуль собирает подробные сведения о состоянии удалённого сервиса. Может применяться при тестировании или мониторинге: - Функции для получения usage: CPU %, RAM %, свободная память. Возможно, использует `psutil` на сервере. - Функция для тестирования скорости: например, прогнать dummy-вектор через поиск, измерить latency. - Логирование OOM: возможно, в `docs/oom_reports/*.md` эти отчёты сгенерированы частично с помощью этого модуля. Например, `vm_diagnostics_phase2.py` (в `scripts/`) возможно запускал series of tests и собирал результаты. - Может интегрироваться с `/health` эндпоинтом: например, `/health` мог бы возвращать `{"cpu": 70, "mem": 8Gb used}` и `VM.Diagnostics` формирует на основе этого вывод.

В `transport_client.py` импорте видно `vm_diagnostics` – вероятно, `TransportClient` может логировать если задержка более X, вызывает `diag`.

rag/init.py – пакетный импорт

И наконец, `rag/__init__.py` собирает все компоненты для удобства импорта. В докстринге перечисляются основные: `CPUEmbedder`, `VectorStore`, `QueryEngine` (с гибридным поиском и MMR). Возможно, здесь определены:

```
from .embedder import CPUEmbedder, ...
from .remote_embedder import RemoteEmbedder
```

```

from .vector_store import VectorStore
from .remote_vector_store import RemoteVectorStore
from .search_service import SearchService
from .indexer_service import IndexerService
from .query_engine import CPUQueryEngine
from .factory import Factory
...
# Возможно, удобные алиасы:
create_embedder = Factory.create_embedder
create_vector_store = Factory.create_vector_store

```

Так, остальные части программы могут писать `from rag import VectorStore, RemoteVectorStore, Factory, RagException` и т.д., и `rag.create_vector_store(config)`.

Также, `rag/__init__.py` мог обрабатывать инициализацию фабрики: например, на импорт определять `Factory.detect_context()`.

Итоговая картина работы RAG: При запуске на клиенте, фабрика инициализируется в контексте CLIENT, и все создаваемые компоненты – Remote*. На VM – контекст VM, компоненты локальные. Это устраняет проблемы рекурсии и позволяет использовать единый код IndexerService/SearchService с Factory.

Архитектурная и интеграционная карта проекта

Чтобы лучше понять взаимодействие всех компонентов, опишем проект в нескольких плоскостях: функциональной, компонентной (архитектурной) и интеграционной.

Функциональный поток данных (Pipeline)

1. **Запуск анализа:** Пользователь через UI или CLI указывает репозиторий. Например, в UI выбирается папка и нажимается "Начать анализ".
2. **Сканирование и подготовка чанков:** Локально `file_scanner.py` собирает файлы, затем `code_chunker.py` разбивает их на чанки. Дополнительно, может быть выводится промежуточная информация (например, "Найдено 50 файлов, сформировано 314 чанков").
3. **Индексация на удалённом сервисе:**
4. Клиент (UI/Analyzer) вызывает `RemoteVectorStore.index_documents(chunks)`.
5. Эта функция упаковывает чанки в HTTP-запрос (POST /index). Возможен предварительный разрез на батчи (например, по 256 чанков).
6. **Передача по сети:** JSON с батчем чанков отправляется на VM (адрес из конфигурации, например `http://10.61.11.54:8000/index`).
7. На VM FastAPI `vm_rag_service` принимает запрос и передаёт данные в `IndexerService.index()`.
8. IndexerService вызывает локальный `CPUEmbedder` – модель генерирует эмбединги для каждого чанка. Затем IndexerService сохраняет эти векторы в Qdrant через `VectorStore.index_documents`.
9. По завершении каждого батча IndexerService может логировать "Эмбединг батча завершён" и "Индексирование батча завершено". После всех батчей – возвращает суммарно количество добавленных.
10. FastAPI отправляет ответ клиенту (например, `{"indexed": 314}`).

11. Клиентский RemoteVectorStore получает ответ и возвращает его в main/UI. UI отображает "Индексировано 314/314 чанков за X секунд".
12. **Задачи пользователя – поиск/вопросы:** Пользователь может задавать вопрос о коде в UI или запрос на поиск.
13. UI вызывает, например, метод `query_engine.search(query_text)` (или напрямую remote search).
14. Клиент формирует запрос к VM: либо напрямую `/search` (передавая `query_text`), либо сначала запрашивает embed vector для текста (через `/embeddings`) и затем `/search` с полученным вектором. Скорее, реализовано одним запросом: `/search` с текстом (чтобы не гонять лишние шаги).
15. На VM `SearchService.search` обрабатывает: embed запроса (вызов CPUEmbedder), поиск векторов (VectorStore.search) и формирование результатов (с применением RRF/MMR, если настроено).
16. Результаты (список чанков с метаданными) возвращаются клиенту.
17. UI отображает найденные кодовые фрагменты.
18. **Генерация ответа GPT:** Если предусмотрено (например, кнопка "Сформировать отчёт"), UI берёт текст вопроса + топ-N найденных чанков, и вызывает `OpenAIIntegration.generate_answer(question, context)`.
19. Эта функция (в `openai_integration.py`) формирует промпт: содержит, к примеру, "Вот файл X, функция Y doing Z: <код>; ... Вопрос: ...". Она обращается к OpenAI API (через internet if key provided).
20. Полученный ответ (например, "Основные классы: ...") возвращается и показывается пользователю.
21. (Дополнительно, система может сохранять этот ответ в файл Markdown – возможно, `main.py` делает это для финального отчёта).
22. **Завершение работы:** Пользователь получает либо интерактивные ответы, либо сформированный отчёт. Репозиторий проанализирован и индекс (в Qdrant) можно сохранить для повторного использования (в Qdrant коллекция остаётся, так что при следующем запуске можно не переиндексировать, если не было изменений – но это будущее улучшение, сейчас, скорее всего, всегда индексирует заново).

Компонентная архитектура (слои и взаимодействие)

• User Interface (Presentation layer):

- *Streamlit Web UI* (`web_ui.py`) – интерактивный фронтэнд для запуска анализа и задавания вопросов. Предоставляет удобный интерфейс, но всю тяжелую логику делегирует нижележащим слоям.
- *CLI Launcher* (`unified_launcher.py`, `main.py`) – альтернатива UI, позволяет через консоль выполнить то же (или скрипт на сервере без UI). `Main.py` тут выступает и как бизнес-логика, и как точка входа.

• Core Logic (Application layer):

- *RepositoryAnalyzer* (логика `file_scanner` + `code_chunker` + *orchestration in* `main.py`) – отвечает за подготовку данных для поиска. Именно здесь происходит сбор и разбиение кода.
- *OpenAIIntegration* (в `openai_integration.py`) – генерация текстовых выводов из результатов поиска. Это связь с внешним LLM-сервисом.
- *Query Engine* (`query_engine.py` на клиенте) – координирует выполнение поискового запроса: обращается к RAG-сервису (или локально, если сконфигурировано) и обрабатывает результаты (кэширование, ранжирование).

- **RAG System (Semantic Search layer):**

- *Embedder*: На клиенте представлен `RemoteEmbedder`, на сервере – `CPUEmbedder`. Они скрыты за фабрикой, поэтому вызывающий код обычно не различает их. Embedder реализует интерфейс создания векторов из текста.
- *Vector Store*: На клиенте – `RemoteVectorStore` (HTTP-клиент к хранилищу), на сервере – `VectorStore` (обёртка над Qdrant). Отвечает за хранение и поиск векторов. `InMemoryVectorStore` – альтернативная реализация для тестов/офлайна.
- *Factory*: Центральный компонент, который связывает вышеперечисленное – при создании RAG-компонентов он подменяет реализации в зависимости от окружения (VM/Client). Это позволяет использовать единый код сервисов на VM и вызывать их из клиента прозрачно.
- *RAG Services (на VM)*: `IndexerService` и `SearchService` – логика обработки `/index` и `/search` соответственно. Они используют Embedder и VectorStore (через Factory) для выполнения своих задач и содержат дополнительные алгоритмы (batching, hybrid rank, etc.).
- *Circuit Breaker & Retry*: Эти утилиты пронизывают RAG слой, обеспечивая отказоустойчивость. `CircuitBreaker` следит за состоянием сервиса, `RetryPolicy` автоматически повторяет операции. Например, `TransportClient` может иметь декоратор `@retry` для запросов.

- **External Integration (Infrastructure layer):**

- *FastAPI application* (`vm_rag_service.py`) – веб-сервер на VM, предоставляющий API для индексации/поиска/эмбединга. Он инициализирует RAG-компоненты локально при старте.
- *Vector Database (Qdrant)*: запущена на VM (возможно, Docker-контейнер). С ней общается `VectorStore` либо по gRPC, либо REST. Qdrant хранит индекс векторов на диске, поддерживает поиск с фильтрацией. Параметры HNSW и квантования настроены для CPU-оптимизации.
- *Embedding Model (Jina v3 / Transformers)*: на VM загрузка модели для `CPUEmbedder`. Это может быть HuggingFace модель (например, 'sentence-transformers/all-MiniLM-L12-v2') или Jina AI модель (даже название v3). Она использует 4 потока (TORCH/OMP/MKL) и ~570M параметров, что указывает на достаточно крупную модель (возможно, модель CodeBERT или аналог).
- *OpenAI API*: вызывается из клиентской части (`OpenAIIntegration`) через интернет. Требуется API ключ, который хранят в `.env` (в `.env.example` вероятно `OPENAI_API_KEY=...`). Ответы GPT используются для финального вывода.
- *Local File System*: хранит код (входные данные) и генерирует отчёты (выход). Также, на стороне VM, Qdrant хранит индексные файлы (db PATH) – настройка `MMap=true` говорит, что Qdrant использует memory-mapped файлы на диске.

Ниже представлено условное **структурное описание**, показывающее компоненты и их взаимодействие:

- **Client (локально):**

- UI (Streamlit)
 - ↳ **RepositoryAnalyzer** (Scanner + Chunker)
 - ↳ **QueryEngine** (Remote calls)
 - ↳ RemoteEmbedder -- HTTP -->
 - ↳ RemoteVectorStore -- HTTP -->
 - ↳ **OpenAI Integration** -- API --> OpenAI GPT
- Factory (знает, что клиент, направляет к Remote...)

- **Server VM (удалённо):**

- FastAPI (`vm_rag_service`) с эндпоинтами
 - ↳ **IndexerService**
 - ↳ Factory (контекст VM: отдаёт локальное)
 - ↳ CPUEmbedder (модель Jina/Transformers)
 - ↳ VectorStore (Qdrant client) -- RPC --> Qdrant DB
 - ↳ **SearchService**
 - ↳ Factory (Embedder + VectorStore локальные)
 - ↳ CPUEmbedder (вектор запроса)
 - ↳ VectorStore.search (поиск по Qdrant)
 - ↳ *SparseEncoder* (генерирует sparse для hybrid, если нужно)
 - ↳ RRF + MMR (ранжирование результатов)
 - ↳ **Health checks / Diagnostics**
- Qdrant (DB) – хранит вектора
- Jina Embedding Model – загружена в памяти

(Стрелки “-->” обозначают вызовы/запросы между компонентами.)

Интеграция и масштабирование

Проект интегрируется с несколькими внешними системами, и есть планы расширения: - **OpenAI & LLMs:** Нынешняя реализация использует один LLM (OpenAI GPT-4 или 3.5) для генерации текста. Масштабирование может подразумевать возможность подключения локальной модели (например, LLaMA) или использование нескольких моделей (см. далее про 2 модели эмбедингов). - **Две модели эмбедингов параллельно:** Идея разработчика – использовать сразу две модели для построения эмбедингов. Это может улучшить качество поиска, особенно если модели разной природы (например, одна оптимизирована под кодовые структуры, другая под семантику текста комментариев). Чтобы это внедрить: - Нужно изменить *Embedder*: вместо одного вектора размерности 1024 генерировать два (или один удвоенной размерности). Например, Model A (768-дим) + Model B (768-дим) -> конкатенация в 1536-дим вектор. Либо хранить два отдельных вектора и модифицировать поиск (делать два поиска и объединять). - Параллельное выполнение: можно загружать две модели и эмбеждать одновременно (в разных потоках или процессах) для скорости. Однако, нужно учесть рост нагрузки: 2 модели = больше потребление памяти (вдвое). - *VectorStore* поддерживает мульти-векторы? Qdrant на 2025 год поддерживает только один вектор на точку, но можно хранить несколько коллекций – например, коллекция_A с эмбедингами модели1, коллекция_B – модели2. Тогда при поиске получать кандидатов из обеих и объединять (аналогично RRF). - Менее затратно – использовать одну модель как основную, вторую только для rerank: например, сначала быстрая модель ищет кандидатов, потом более точная модель пересчитывает их расстояния. Это уменьшает индекс во 2-й модели. - Рекомендация: начать с простого – конкатенировать векторы и загрузить их в единый индекс (требуется переиндексация). Или выполнять два поиска и делать RRF. В архитектуре придется модифицировать *IndexerService* (чтобы вызывал оба CPUEmbedderA и B), *SearchService* (чтобы комбинировал результаты). - **Масштабирование на несколько VM:** Пока система рассчитана на одну VM. В будущем, можно разделить: например, одну VM держать для эмбедингов (модель), другую – для базы (Qdrant) и API. Jina Framework, возможно, позволяет распределить (подобно microservice). Уже сейчас Qdrant – отдельный сервис, можно выносить на другой хост с небольшими изменениями (настрой хоста). - **Оптимизация больших репозитория:** Документы OOM-репортов показывают проблемы при больших партиях. Исправлено увеличением таймаутов и квантованием, но можно и дальше оптимизировать: - Стриминг индексации: не держать весь батч эмбедингов в памяти, а индексировать по мере

готовности (pipeline chunk->embed->index поточно). - Ограничение параллелизма: сейчас config embed_workers=4, search_workers=4, но это потоки на клиенте, а на сервере – FastAPI uvicorn воркеры и torch threads. Баланс этих параметров может быть отрегулирован для лучшего throughput (например, позволить 2 одновременных поиска, но не более, иначе память кончается). - Мониторинг: интегрировать vm_diagnostics, чтобы до OOM не доводить – например, если осталось <X MB памяти, отклонять новые индексации (CircuitBreaker pattern). - **Удалённые парсеры:** Упомянулось, что парсеры модулей были вырезаны. Возможно, план – вынести парсинг и chunking в отдельный сервис или использовать готовые парсеры (например, tree-sitter). Сейчас chunker, видимо, простой; улучшение парсеров повысит качество chunking (например, лучше определять границы функций).

Анализ несоответствий и потенциально мёртвого кода

Проект находится в активной фазе рефакторинга, поэтому есть несколько мест, где код и документация могут diverge, а также куски кода, утратившие актуальность:

- **Разнородные реализации HTTP-клиентов:** Как отмечено, *RemoteEmbedder* использует асинхронный aiohttp через TransportClient, тогда как *RemoteVectorStore* изначально был реализован иначе (скорее всего, через requests). Это привело к усложнению кода (нужно поддерживать оба метода) и багам (пропущенные таймауты, ошибки с coroutine). Рекомендуется унифицировать: выбрать единый подход. Предпочтительнее – асинхронный (aiohttp) с общим session, так как FastAPI тоже async. Если *RemoteVectorStore* перевести на TransportClient, можно убрать лишний код и гарантировать, что везде выставлены таймауты и retries по единому месту. Это повысит надёжность: например, метод `TransportClient.post_json('/index', data)` с встроенным `timeout=config.timeout_seconds` решит проблему зависания поиска. Сейчас же отсутствие timeout в одном месте (session.post) приводило к зависанию UI на неопределённое время – проблему, которую нужно устранить.
- **Использование `asyncio.to_thread` и определение coroutine:** В баге 1 индексации было явное проявление того, как опасно неправильно распознавать асинхронность. Там async функция была вызвана через to_thread, что не выполняло её. После исправления система заработала (0/260 -> нормально индексирует). Вывод: убедиться, что все async функции вызываются с `await`, а все блокирующие – через to_thread. Вероятно, EventLoopManager и фабрика решили большую часть таких коллизий. Тем не менее, нужно проверить, нет ли где-то ещё неоптимального использования. Тесты *test_async_sync_fix.py* и *test_fixes_simple.py* должны проходить, но если появляются новые корутины, нужно сразу следовать шаблону (например, always use run_async_safe for remote calls).
- **Прототипы и дублирующие модули (мёртвый код):** `context_prototype.py` и `factory_prototype.py` по сути мёртвый код, не используемый в продакшн-пути. Их наличие может сбивать с толку. Например, разработчик решивший править context, может открыть `context_prototype.py` и внести изменение, которое ни на что не повлияет. Лучше удалить или пометить явно как Deprecated. То же касается *vm_start.py.backup*, *config.py.backup*, *vm_rag_service.py.backup* – эти файлы, вероятно, бэкап старого состояния перед большими правками. Их не стоит держать в репозитории надолго. Все важные изменения уже отражены в md-документах и git истории, так что backup-файлы можно убрать. Это снизит путаницу и облегчит сопровождение (и небольшой выигрыш: не будут случайно импортированы).

- **Placeholder и хранение контента:** Был упомянут случай, когда поиск возвращал `placeholder` вместо текста чанка. Судя по коду, `placeholder` используется `InMemoryVectorStore` для контента заглушек. Если по какой-то причине система обращалась к `InMemory` вместо `Qdrant`, то понятно, почему текст не находился. Например, если `VM`-сервис упал, а `QueryEngine` ловит `VectorStoreConnectionError` и решает использовать `fallback = InMemory` (которая пустая, с заглушками) – тогда поиск ничего осмысленного не даст. Неясно, реализован ли такой `fallback`, но `settings.json` имеет `enable_fallback: true` для `chunking`, а про `fallback` для поиска не сказано. Другой сценарий: возможно, решено **не хранить полный content** каждого чанка в `Qdrant payload` (чтобы не раздувать БД). Тогда `placeholder` может реально быть сохранён вместо `content`. В `MemoryVectorStore` примере `'content': 'placeholder'`. Если `SearchService` просто возвращает то, что в `payload (placeholder)`, `UI` получит пустышку. Однако, умный вариант – хранить `content` отдельно (например, в файлах), а по идентификатору чанка затем подтягивать с диска оригинальный текст при отображении. Возможно, это план на будущее. В текущей реализации, скорее всего, `content` всё же сохраняется (иначе `GPT` не сможет ответить). **Рекомендация:** проверить, что `payload` содержит нужный текст или ссылку. Если `placeholder` где-то проскакивает – убрать его использование или заменить на подгрузку реального текста перед выдачей. Например, `QueryEngine` мог бы по результатам поиска открыть исходный файл и вытащить соответствующий кусок текста (зная `file_path` и границы). Это гарантирует актуальность и снимает дублирование хранения. Поскольку проект и так имеет доступ к файловой системе, это несложно.

- **Таймауты и зависания:** После увеличения таймаута до 1 часа проблема зависаний в больших индексах решена, но негативный эффект – теперь при реальной проблеме (например, сервис завис и не отвиснет) `UI` будет ждать час. Нужно подумать: 3600с – очень долго для пользователя. Возможно, стоит реализовать **прогресс-индикатор** или **heartbeat**: например, клиент каждые 60с опрашивает `/health` или получает от сервиса промежуточные ответы. В `FastAPI` можно сделать `streaming response (Server-Sent Events)` для прогресса. Пока не реализовано, но упоминание *“sock_read hang”* говорит, что был случай, когда сервис завис и пользователь не понимал, что происходит. `CircuitBreaker` частично спасает, но он сработает только после `n` ошибок. **Рекомендация:** добавить более короткий **socket timeout** уровня `TCP`, и делать периодические `keep-alive` проверки. Либо разбить долгий запрос на серию коротких (что собственно батчи и делают; можно после каждого батча присылать промежуточный ответ – но `HTTP 1.1` не поддерживает несколько ответов на один запрос). Другой подход – на `UI` после определённого времени отображать кнопку “Отмена” или предложение перезапустить.

- **Логирование и мониторинг:** Для стабильности важна прозрачность работы. Проект уже ведёт логи (`logging` используется), и есть `cleanup.log`, `oom_reports`. Следует продолжать:

- Логировать все вызовы к внешним системам с временем выполнения. Например, `"Indexed batch of 100 chunks in 30s"`.

- Логировать отклонённые запросы (`CircuitBreaker open`).

- Метрики: использовать `collect_vm_metrics.py` регулярно (`cron`) или включить `/health` выдавать `CPU/RAM`, чтобы мониторинг мог оповестить если память почти исчерпана (чтобы админ смог перезапустить сервис до `OOM`).

- **Technical Debt из документа:** Вероятно, там упомянуто, что нужно: унифицировать конфигурацию (возможно, некоторые переменные дублируются между `.env` и `settings.json`), доработать тесты (например, добавить интеграционный тест: запустить полный цикл

индексации и запроса на небольшом репо). По возможности, уже стоит удалить dead code, как обсуждали, и чистить backups.

• **Тесты:** В репозитории есть несколько тест-файлов (`test_async_sync_fix.py`, `test_factory_prototype.py`, `test_fixes_simple.py`, `test_chunker_diagnosis.py`). Они указывают на конкретные известные проблемные зоны. Например:

- `test_async_sync_fix.py` – вероятно проверяет, что вызовы `RemoteVectorStore/Embedder` работают без исключений `async`.
- `test_fixes_simple.py` – может, проверяет, что баги 1-3 действительно исправлены (например, что `index_documents` возвращает число `>0`, а не `coroutine`, что `web_ui chunk_type` доступен).
- `test_chunker_diagnosis.py` – возможно, содержит кейсы на `chunker` (вход: кусок кода – выход: список чанков, сравнение с ожиданием).
- `test_contract_implementation_plan.md` – это документ, но возможно, с указанием как улучшить тестовый контракт (возможно, `rewrite tests to not rely on internal details`).

Чтобы гарантировать качество, нужно эти тесты регулярно запускать (CI). В `.github/ci.yml` они вероятно подключены.

Вывод о мёртвом коде: Подтвержденными “мёртвыми” модулями являются `factory_prototype.py`, `context_prototype.py` и связанные с ними тесты (если новые `Factory` и `Context` полностью заменили их). Также, возможно, `context.py` не используется нигде и функционал контекста сейчас встроен в `QueryEngine` – тогда `context.py` тоже становится кандидатом на удаление или задействие. Проверка использования показала, что `context.py` не импортируется, значит, его код не выполняется – это технический долг (реализованная, но не интегрированная функциональность).

Кроме того, `utils.py` нужно проверить: может какие-то утилиты там не используются (например, если `safe_*` функции переехали). `openai_integration.py` надо убедиться, что код (`OpenAIIntegration` class) действительно вызывается – вероятно, `main` или `UI` вызывает, но если нет – тогда часть его неиспользуемая. Однако, судя по цели проекта, `OpenAIIntegration` точно нужен для финального отчёта.

Рекомендации по оптимизации и развитию

На основе проведённого анализа можно сформулировать ряд рекомендаций, фокусируясь на масштабировании, оптимизации и устранении причин прошлых багов:

1. **Унифицировать механизмы удалённых вызовов:** Сейчас для `embedder` применяется асинхронный `TransportClient` с `aiohttp`, а для `vector_store` – видимо, синхронный `requests`. Стоит привести `RemoteVectorStore` к аналогичной асинхронной реализации. Это позволит централизованно управлять таймаутами и ретраями, а также снизит нагрузку (`aiohttp` лучше переиспользует соединения, не создает новый TCP на каждый запрос). После унификации – удалить избыточный код (например, `utils.session.post` если был, или `to_thread`).
2. **Гарантировать таймауты на всех сетевых операциях:** Проверить, что **каждый** HTTP запрос имеет явный `timeout`. В частности, добавить `timeout` в `RemoteVectorStore.search` (тот самый пропуск). Неплохо установить и отдельный таймаут на соединение (`connect_timeout`) и на чтение (`sock_read_timeout`), например, 10с и 50с, чтобы

быстрее обрывать, если сеть недоступна. Длительный `timeout_seconds=3600` оставить как верхний bound, но лучше обработать прогресс (см. пункт 3).

3. **Реализовать обратную связь в длительных операциях:** Часовой таймаут решает проблему обрыва, но UX от этого страдает. Следует подумать о **progress updates**:
4. Можно модифицировать `/index` endpoint на сервере: сразу после получения запросов запускать индексацию, а клиенту периодически слать статус (в WebSockets или Server-Sent Events). Streamlit может читать такие обновления и обновлять progress bar. Или проще: разбить индексирование на куски и вызывать многократно `/indexBatch` – тогда UI сможет после каждого батча обновляться (например, 256 чанков проиндексировано – отобразить, затем следующий запрос).
5. В текущем коде, хотя батчи используются, они скрыты за одним `/index`. Возможно, лучше API `/index` сделать идемпотентным для батчей: т.е. клиент сам посылает 256 за раз в цикле. Тогда после каждого можно обновлять UI и, если один из батчей упал, повторить его без пересылки уже успешных.
6. Для `/search` тоже применимо: если поиск более комплексный (dense + sparse), но он обычно короткий (<1с), так что тут достаточно просто timeout.
7. **Очистка неиспользуемого кода:** Как часть оптимизации поддержки – удалить или пометить `prototype` модули. Поддерживать два параллельных набора классов ни к чему. Фабрика уже внедрена и протестирована, можно смело удалять старый код. То же с context: если новая логика не требует отдельного класса context, удалить старый модуль.
8. Backup-файлы (.backup) лучше убрать из репозитория (они хранятся, видимо, для сравнения, но для этого есть система контроля версий).
9. Tests: удалить `test_factory_prototype.py` если сам `prototype` ушёл. Взамен, можно добавить больше тестов на актуальные `Factory`, `IndexerService`, `SearchService` (возможно, расширить `test_fixes_simple.py`).
10. **Улучшить хранение контента чанков:** Чтобы снизить память и гарантировать актуальность:
11. **Не хранить полный текст в Qdrant:** вместо этого хранить ссылку (`file_path + line_range`). При выводе результата, UI или `QueryEngine` могут прочитать соответствующий кусок из файла напрямую. Это избавит от дублирования (код уже есть на диске, нет смысла дублировать в БД), и уменьшит размер payload, ускоряя поиск.
12. Если необходима некоторая часть контента для семантики (например, sparse search или LLM контекст), можно хранить только короткий превью (первые 200 символов).
13. Проверить, что placeholder не просачивается: может, совсем убрать его – лучше хранить пустую строку или не хранить content поле вообще, чем 'placeholder'. Иначе можно случайно вывести пользователю "placeholder". Если content отсутствует, UI мог бы сам догрузить его, зная файл и границы.
14. **Две модели эмбедингов – план реализации:**

15. Начать с экспериментов: подключить вторую модель и сравнить качество поиска. Например, сделать опционально: `USE_SECOND_MODEL=True` в config, и если включено – добавить вторую модель в CPUEmbedder (или отдельный класс).
16. Самый простой путь: конкатенация векторов. Нужно убедиться, что Qdrant поддерживает увеличенную размерность (в config EMBEDDING_DIMENSION нужно поставить новую сумму). Qdrant без проблем хранит большие вектора, вопрос в производительности (но $1024+1024=2048$, это нормально).
17. Альтернативно, хранить два индекса: можно создать вторую коллекцию (code_chunks_modelB). Тогда *IndexerService* индексирует в обе, *SearchService* делает два поиска и RRF на выходе. Это больше кода, но позволяет независимо настроить вес каждого канала.
18. Принять решение, как параллелить: можно просто делать `vecA = modelA(texts); vecB = modelB(texts)` последовательно (попроще, меньше потоков, но дольше) или с помощью `asyncio.gather` / `ThreadPool` – параллельно (в 2 потока CPU – но тут надо осторожно, обе модели могут одновременно потреблять RAM). Можно, например, если RAM позволяет, а CPU многоядерный, то параллельно.
19. Протестировать на реальных запросах: удостовериться, что комбинированный индекс действительно улучшает поиск (возможно, вторая модель должна быть чем-то отличающейся значительно, иначе толку мало).
20. **Продолжить оптимизацию производительности VM:**
21. **Квантование** уже используется (SQ – scalar quantization), можно оценить переход на PQ (product quantization) для ещё большего снижения памяти, если точность позволяет.
22. **HNSW parameters:** M=24, ef=256 – можно подстраивать: больший ef_search (например 512) повысит точность поиска, ценой CPU. Можно сделать ef_search динамическим: высокий при индексации/первом полном анализе, и поменьше при быстрых запросах.
23. **Шардирование:** Qdrant поддерживает sharding по узлам. Если планируется рост данных (больше, чем поместится на одной VM), можно предусмотреть, как добавить вторую VM с Qdrant и распределить коллекцию. Пока, видимо, не актуально (репозитории обычно не миллионы документов, хотя код может генерировать >100k чанков в крупных проектах).
24. **Обработка больших файлов:** Если попадётся файл на сотни тысяч строк (большие JSON, SQL дампы), chunker может съесть много памяти пытаясь держать его полностью. Возможно, стоит добавить ограничения (например, не индексировать файлы > определённого размера, либо делить их на части заранее).
25. **Повышение отказоустойчивости:**
26. Настроить **Circuit Breaker** тоньше: например, если VM отвечает 500 (server error) 5 раз подряд – breaker open на 1 минуту. Если Timeout – open на 5 минут (так, чтобы сервис успел восстановиться). Но если Timeout вызван перегрузкой, уже принятие новых запросов через 5 мин может помочь (вдруг разгрузилось).
27. Предусмотреть **фолбэк режим:** Например, если RAG недоступен, система всё равно может работать ограниченно – без семантического поиска. Она может делать простое полнотекстовое сканирование (например, grep по файлам) и выдавать ответы GPT, опираясь только на найденные по ключевым словам куски. Это хуже по качеству, но лучше, чем ничего. *enable_fallback* в settings может касаться этого – если True, можно при недоступности VM выполнить локальный поиск по содержимому файлов (например,

простым строковым совпадением) и кормить это в GPT. Это, конечно, сложнее, но для отказоустойчивости полезно.

28. **Документация и соответствие кода:** Постоянно актуализировать комментарии и md-файлы:

29. После внедрения 2х моделей, описать это в архитектуре, обновить Technical Architecture.md.

30. Удалив старый код, отметить в Technical Debt, что долгов стало меньше

31. Возможно, создать developer guide по тому, как добавить новый тип компонента или как проводить тестирование (чтобы onboarding новых разработчиков был проще).

32. Описание API: неплохо бы иметь swagger или просто docs, описывающие, что принимает/возвращает /search, /index. FastAPI может автоматически генерировать документацию (доступна по /docs).

33. **Тестирование на граничных случаях:** Использовать уже реализованные диагностические средства:

- Рекомендовано протестировать индексацию на различных размерах батчей: 1) очень маленький репо (пару чанков), 2) средний (200-300), 3) большой (800-1000 чанков) и убедиться, что все успешно проходит и UI показывает корректный прогресс.
- Проверить, что CircuitBreaker не срабатывает ложно, например, при коротких сбоях сети – т.е. он должен закрываться обратно после успешных попыток.
- Тесты на поисковые алгоритмы: убедиться, что RRF и MMR работают как задумано. Можно подготовить искусственный набор, где dense и sparse дают разный порядок, и проверить, что RRF выход соответствует формуле.
- Если внедрять вторую модель – обязательно unit-тест: например, сделать две искусственные модели (одна всегда возвращает один и тот же вектор, другая – другой) и проверить, что QueryEngine действительно комбинирует оба результата.

В заключение, проект **repo_sum** имеет сложную, но хорошо структурированную архитектуру, которая сочетает в себе собственный анализ кода и современные подходы семантического поиска. Уже проделан значительный объём работы по исправлению унаследованных проблем (таймауты, рекурсия, синхронизация async) – это подтверждается документированными багфиксами. Остаётся довести рефакторинг до конца, удалив неактуальные части и укрепив согласованность между компонентами. Следуя данным рекомендациям, система станет более масштабируемой (способной работать с ещё большими кодовыми базами и, возможно, несколькими моделями), устойчивой к сбоям и проще в сопровождении. Это заложит основу для выпуска версии 0.6 и дальнейшего развития проекта.
